

Machine Learning

CMPSCI 589

Bruno C. da Silva

bsilva@cs.umass.edu

Reinforcement Learning (part 2/3)

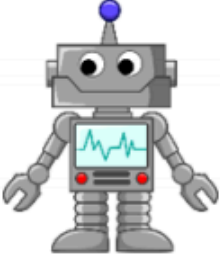
UMassAmherst
College of Information
& Computer Sciences



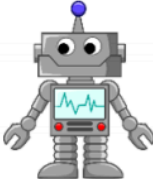
This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/)



Review: 687-Gridworld: A Simple Environment

Start State 1	State 2	State 3	State 4	State 5
State 6		State 8	State 9	State 10
State 11	State 12	Obstacle	State 13	State 14
State 15	State 16	Obstacle	State 17	State 18
State 19	State 20		State 22	End State 23

Review: 687-Gridworld: A Simple Environment

Start State 1	State 2	State 3	State 4	State 5
State 6		State 8	State 9	State 10
State 11	State 12	Obstacle	State 13	State 14
State 15	State 16	Obstacle	State 17	State 18
State 19	State 20		State 22	End State 23

- Agent starts in State 1
- Process ends when agent reaches State 23

State: Position of robot (but not the direction it is facing)

Actions: Attempt Up (AU)
Attempt Down (AD)
Attempt Left (AL)
Attempt Right (AR)

Environment Dynamics:

80% - moves in specified direction

5% - gets confused, veers right of intended direction

5% - gets confused, veers left of intended direction

10% - temporarily breaks and doesn't move at all

If colliding with walls or obstacles, doesn't move

Rewards:

-10 - entering the state with water

+10 - entering the goal state (and process terminates)

0 - entering any other state

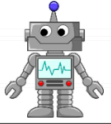
Reward discount parameter: $\gamma = 0.9$

(will be discussed later)

Review: Markov Decision Processes

- Let p be the **transition function**

Describes how the state of the environment changes

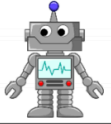
Start State 1	State 2	State 3	State 4	State 5
State 6		State 8	State 9	State 10
State 11	State 12	Obstacle	State 13	State 14
State 15	State 16	Obstacle	State 17	State 18
State 19	State 20		State 22	End State 23

$$p(\text{State9}, \text{AU}, \text{State4}) = \Pr(S_{t+1} = \text{State4} \mid S_t = \text{State9}, A_t = \text{AU}) = 0.8$$

Review: Markov Decision Processes

- Let p be the **transition function**

Describes how the state of the environment changes

Start State 1	State 2	State 3	State 4	State 5
State 6		State 8	State 9	State 10
State 11	State 12	Obstacle	State 13	State 14
State 15	State 16	Obstacle	State 17	State 18
State 19	State 20		State 22	End State 23

$$p(\text{State9}, \text{AU}, \text{State4}) = \Pr(S_{t+1} = \text{State4} \mid S_t = \text{State9}, A_t = \text{AU}) = 0.8$$

$$p(\text{State9}, \text{AU}, \text{State8}) = \Pr(S_{t+1} = \text{State8} \mid S_t = \text{State9}, A_t = \text{AU}) = 0.05$$

Review: Policies

- **Policy**: decision rule - how agent selects actions
maps states to actions (determines behavior of agent)

Current state	Action to take
State1	AR
State2	AU
State3	AL
State4	AL
...	

- Let the function π be a **policy**

Learning: corresponds to agent **changing its policy**

Discounting

- Let γ be the reward discount parameter

$$\gamma \in [0,1]$$

"Will be discussed ~~later~~ now"

Discounting

The Marshmallow Test

“Here's a marshmallow.

*You can either **wait**, and I'll give you another one, so **you'll have two**,
or you can **eat it now**”*



Discounting

The Marshmallow Test

“Here's a marshmallow.

*You can either **wait**, and I'll give you another one, so **you'll have two**,
or you can **eat it now**”*

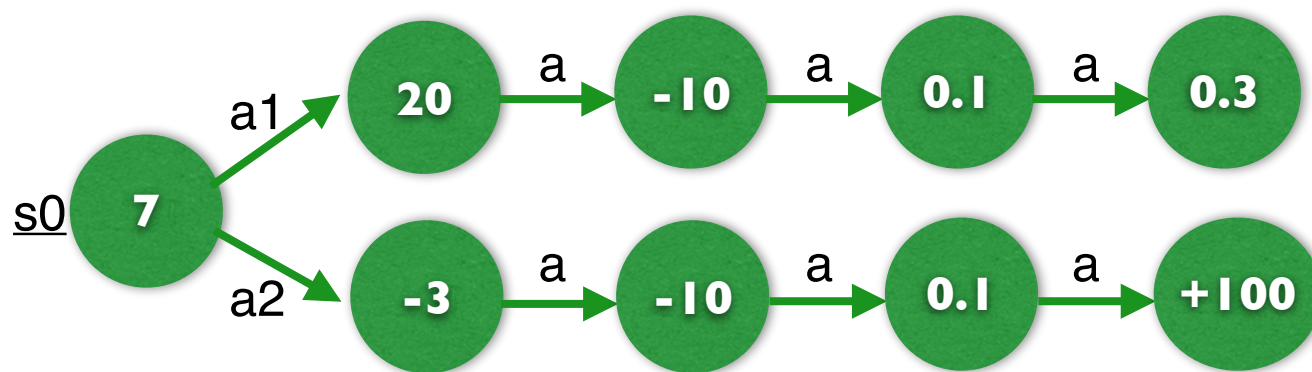
-
- Many people pick one marshmallow *now* when presented with these options
 - Suggests that rewards obtained in the **distant future are worth less** to us
than rewards in the near future
 - Reward discount parameter, γ
 - encodes, in the objective function, discounting of rewards
based on how distant in the future they may occur

Discounting

- Recall that an agent that follows an optimal policy, π^* ,
maximizes the expected total amount of reward

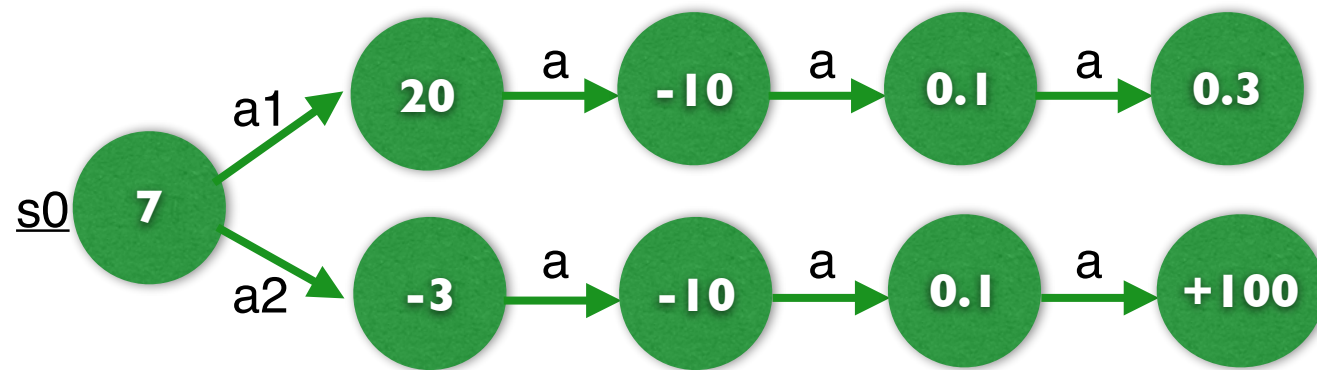
Intuitively, what is a “good” action to take when in state S_t ?

- 1) possibly good “immediate” reward (in the current state)
- 2) but also that takes agent to future states from which
it can collect lots of rewards



which action to take in s_0
to obtain the largest
sum of rewards?

Discounting



Should policy π take action a_1 or a_2 , when in state s_0 ?

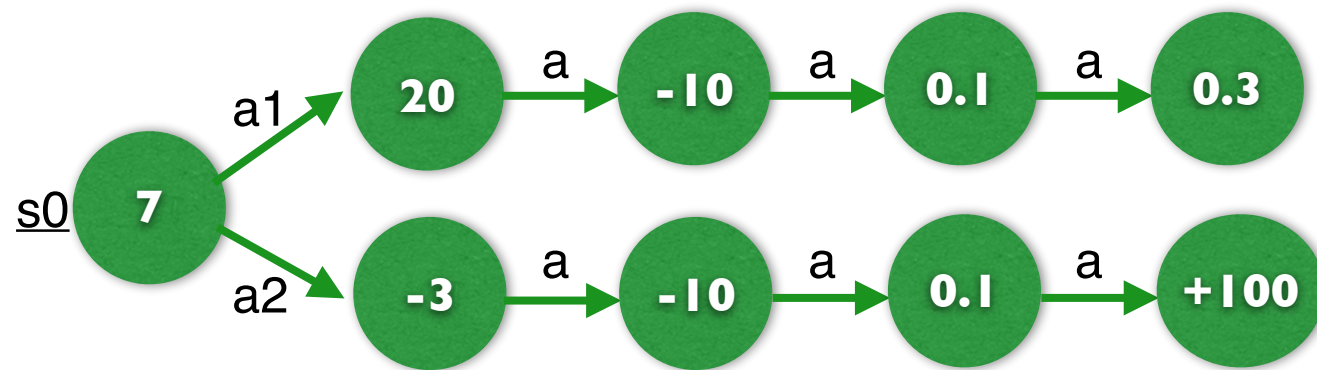
$$J(\pi) = 7 + \begin{cases} a_1 : 20 + (-10) + 0.1 + 0.3 \\ a_2 : -3 + (-10) + 0.1 + 100 \end{cases}$$
$$= 7 + (-3) + (-10) + 0.1 + 100 = 94.1$$

Are rewards a few steps ahead worth the same to us?

Agent could:

- execute a_1 , get a good immediate reward (20) but then keep getting low rewards
- execute a_2 , get low rewards for a while, but then get a really large reward (100)

Discounting



Should policy π take action a_1 or a_2 , when in state s_0 ?

$$J(\pi) = 7 + \begin{cases} a_1 : 20 + (-10) + 0.1 + 0.3 \\ a_2 : -3 + (-10) + 0.1 + 100 \end{cases}$$

$$= 7 + (-3) + (-10) + 0.1 + 100 = 94.1$$

Are rewards a few steps ahead worth the same to us?

$\gamma = 0$?

$\gamma = 1$?

$\gamma = [0,1)$?

$$a_1 : 20 + \gamma(-10) + \gamma^2 0.1 + \gamma^3 0.3$$

$$a_2 : -3 + \gamma(-10) + \gamma^2 0.1 + \gamma^3 100$$

Expected Return:

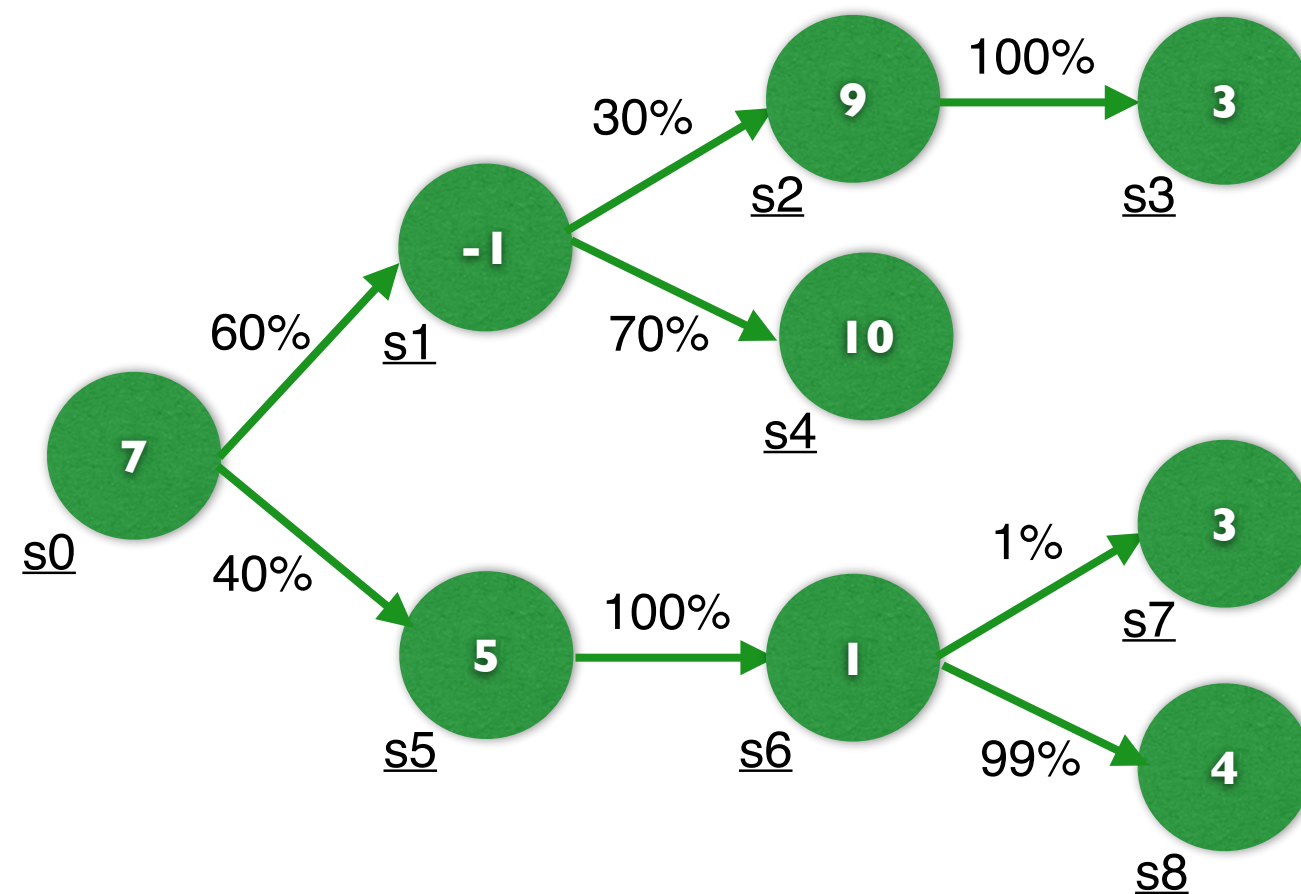
Quantifying the long-term effect of actions

(intuition)

Returns

- Assume the agent is following some policy π . Below, we show:
 - the transition probabilities resulting from following π
 - the rewards for entering each state

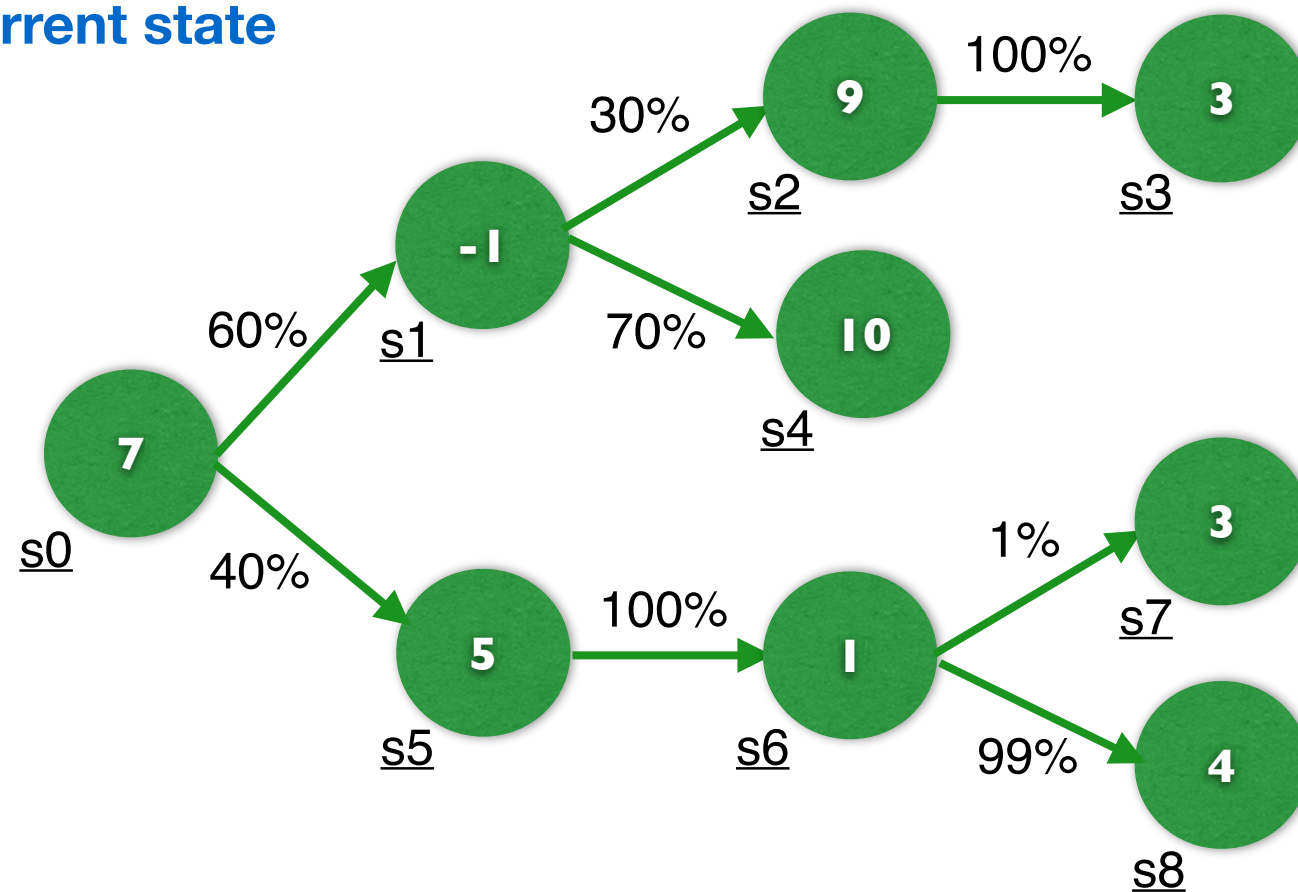
(assume, for simplicity, $\gamma = 1$)



Returns

$$G := \sum_{t=0}^{\infty} \gamma^t R_t = R_0 + R_1 + R_2 + \dots$$

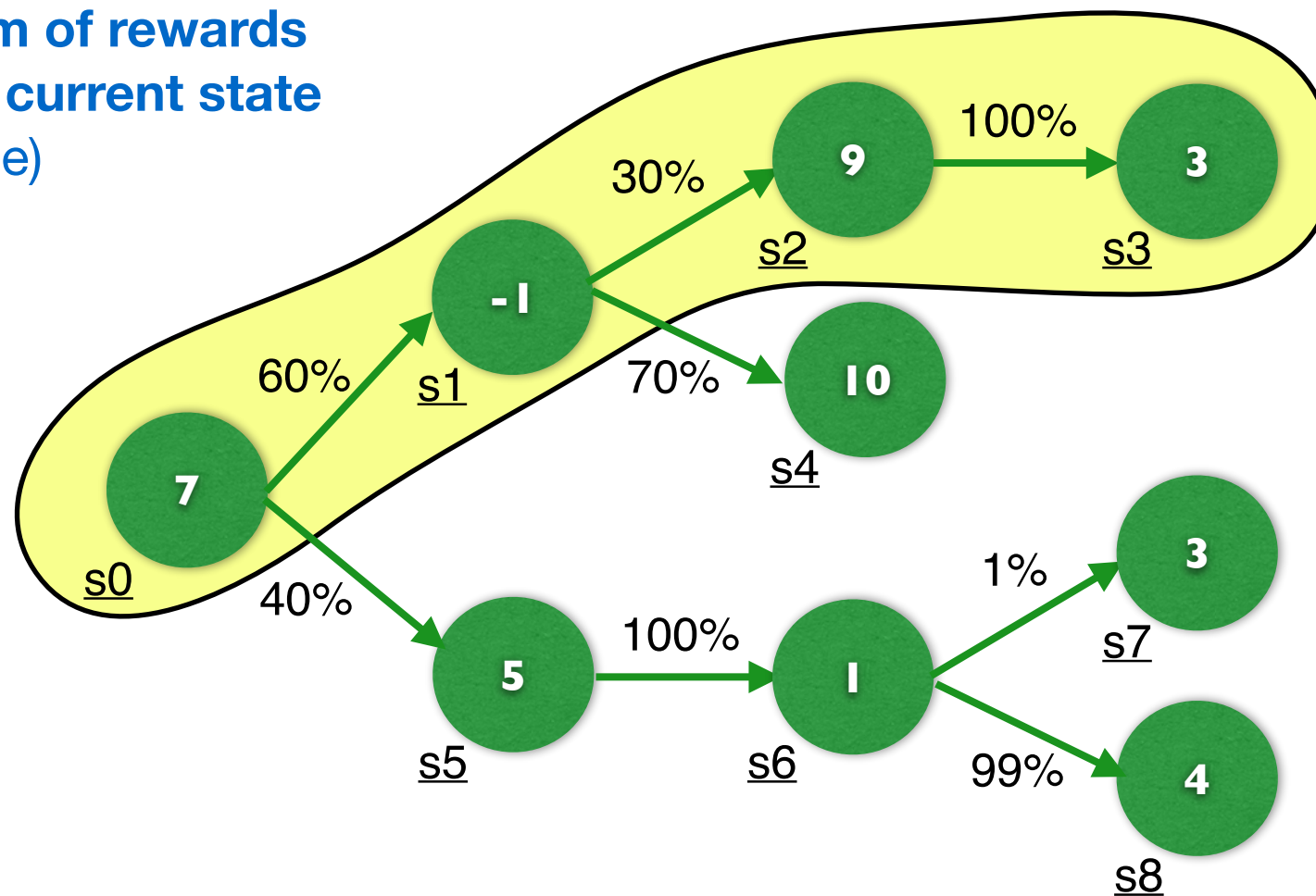
G: one possible sum of rewards
if starting at the current state
(a random variable)



Returns

$$G := \sum_{t=0}^{\infty} \gamma^t R_t = R_0 + R_1 + R_2 + \dots$$

G: one possible sum of rewards
if starting at the current state
(a random variable)



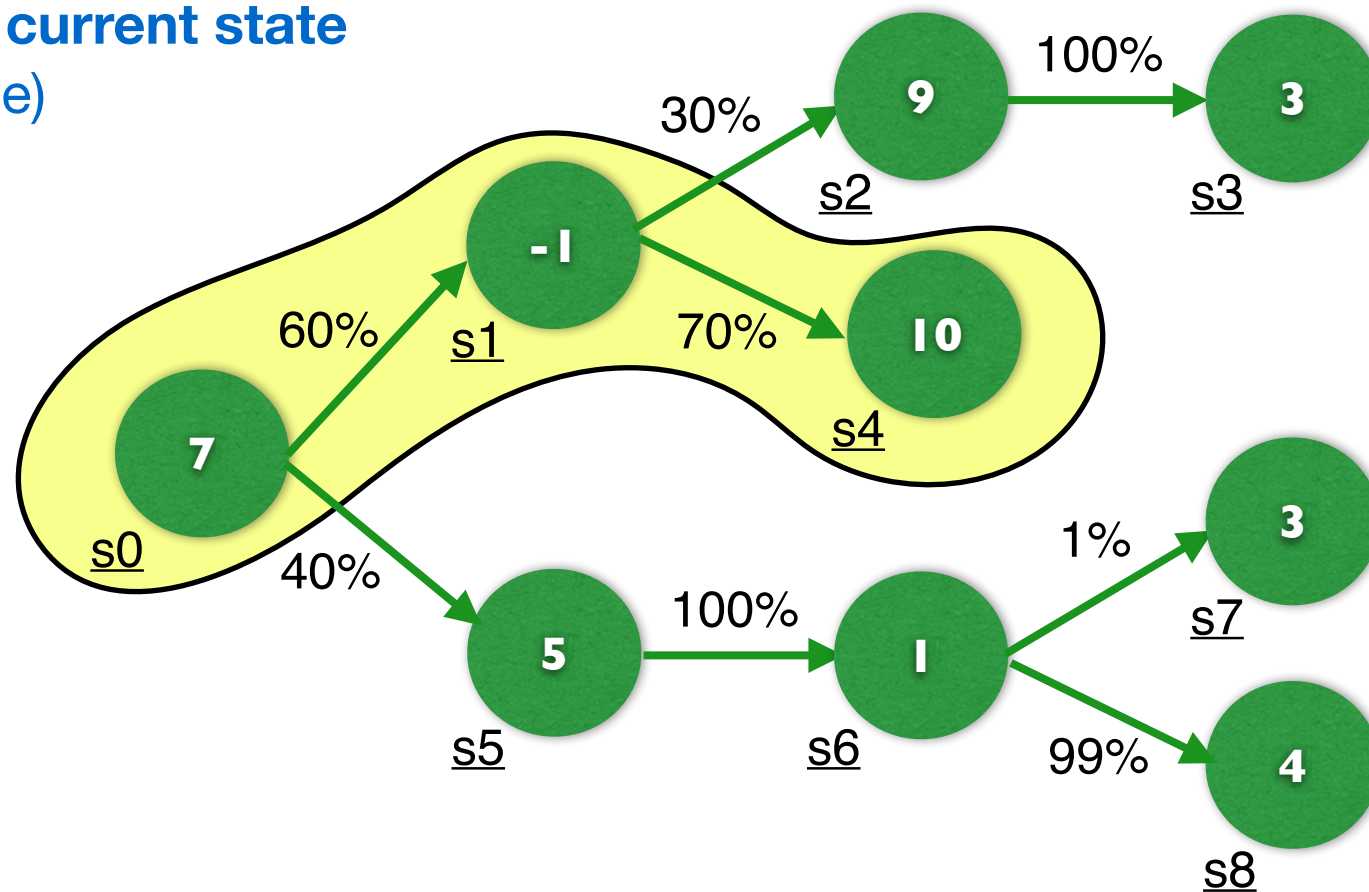
A few possible values of G :

$$G = 7 + (-1) + 9 + 3$$

Returns

$$G := \sum_{t=0}^{\infty} \gamma^t R_t = R_0 + R_1 + R_2 + \dots$$

G: one possible sum of rewards
if starting at the current state
(a random variable)



A few possible values of G :

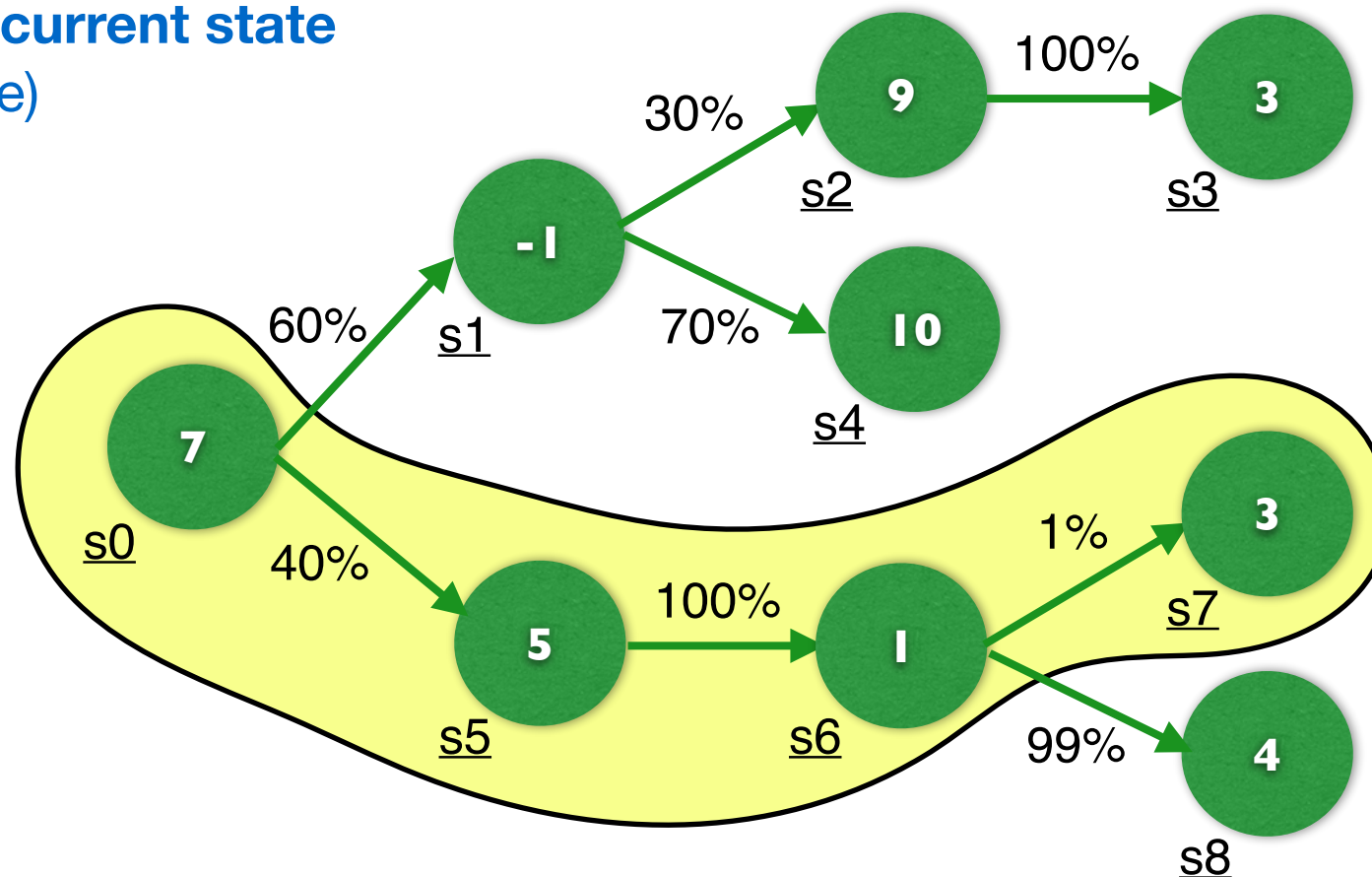
$$G = 7 + (-1) + 9 + 3$$

$$G = 7 + (-1) + 10$$

Returns

$$G := \sum_{t=0}^{\infty} \gamma^t R_t = R_0 + R_1 + R_2 + \dots$$

G: one possible sum of rewards
if starting at the current state
(a random variable)



A few possible values of G :

$$G = 7 + (-1) + 9 + 3$$

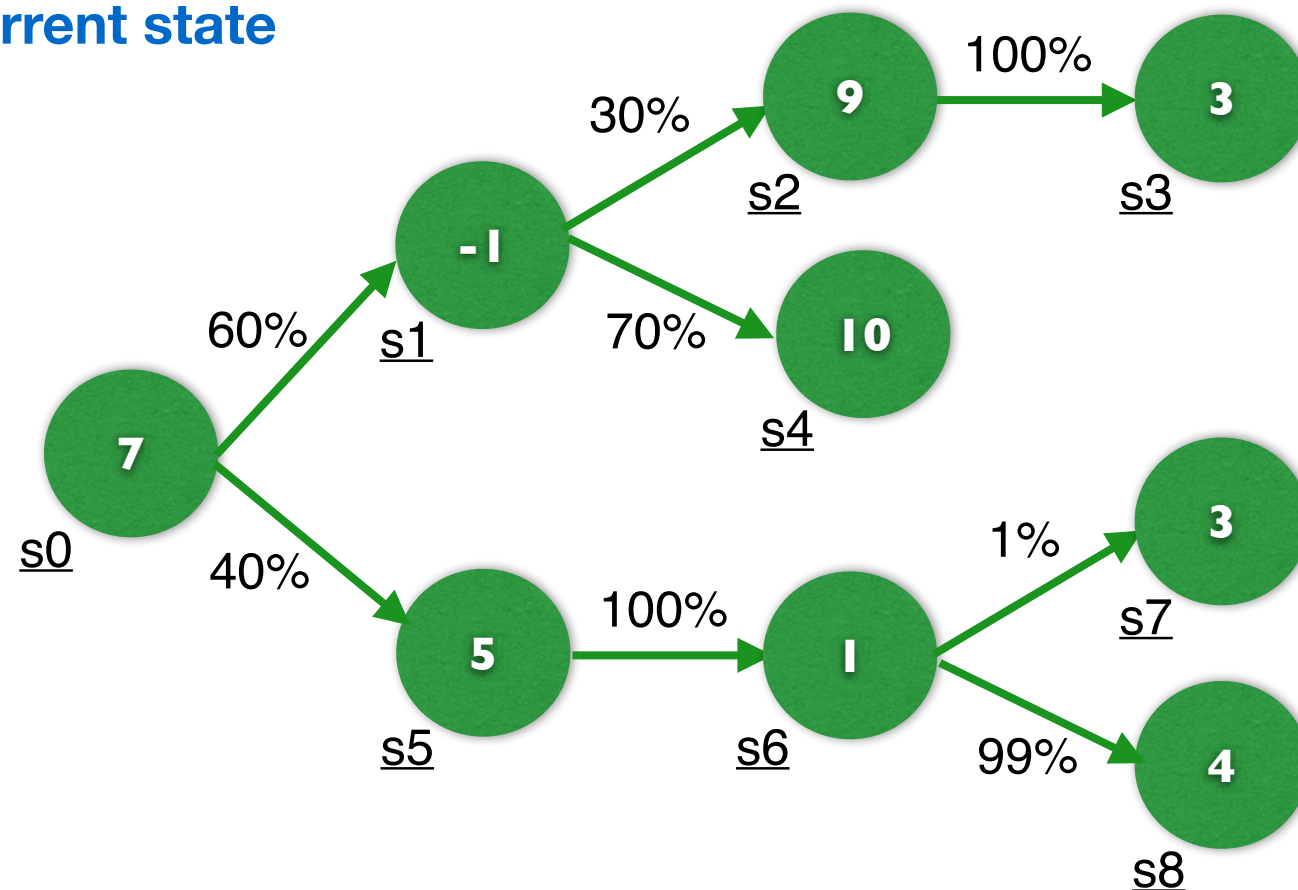
$$G = 7 + (-1) + 10$$

$$G = 7 + 5 + 1 + 3$$

Returns

$$G := \sum_{t=0}^{\infty} \gamma^t R_t = R_0 + R_1 + R_2 + \dots$$

G: one possible sum of rewards
if starting at the current state
(a random variable)



A few possible values of G:

$$G = 7 + (-1) + 9 + 3$$

$$G = 7 + (-1) + 10$$

$$G = 7 + 5 + 1 + 3$$

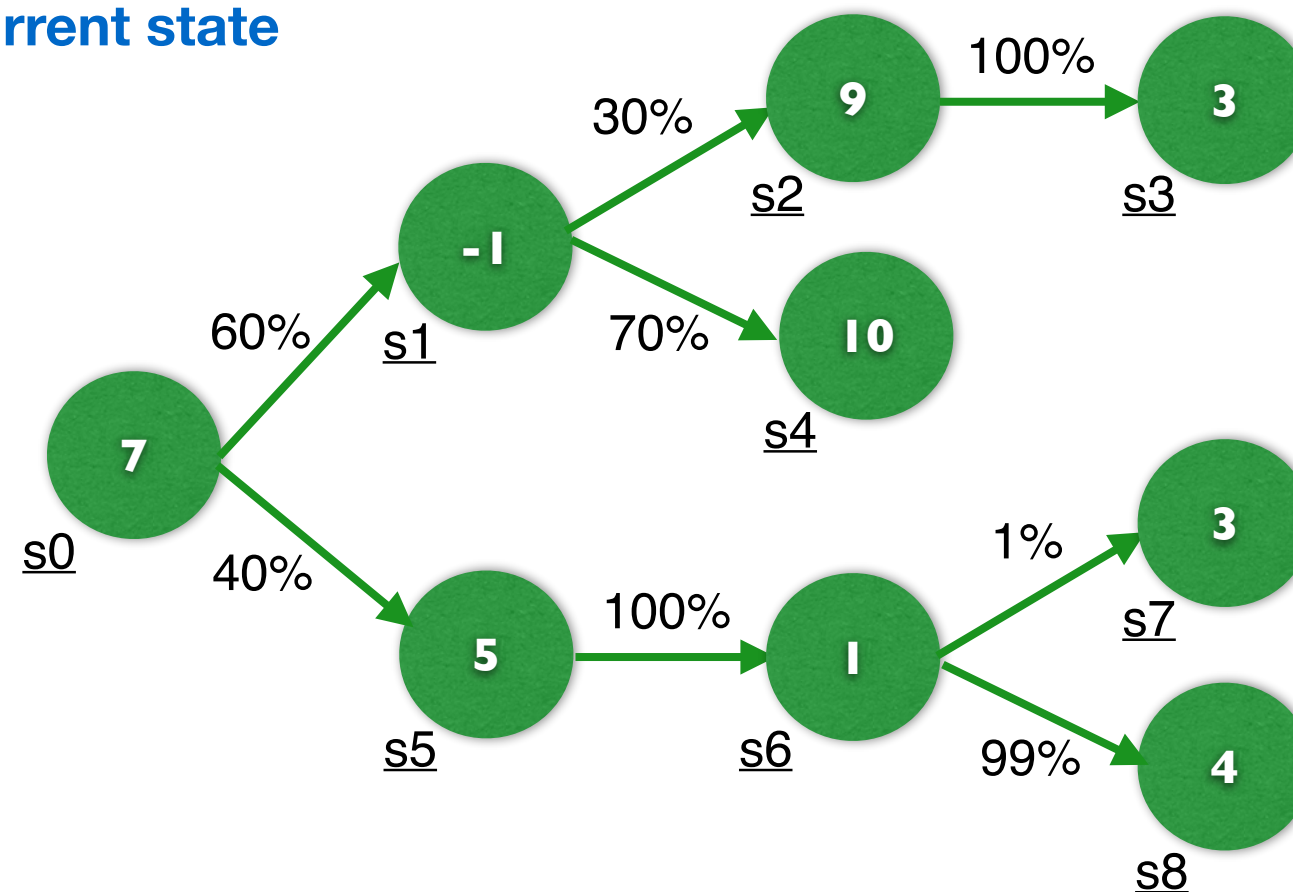
(with what probability?)

(with what probability?)

(with what probability?)

$$G := \sum_{t=0}^{\infty} \gamma^t R_t = R_0 + R_1 + R_2 + \dots$$

G: one possible sum of rewards
if starting at the current state
(a random variable)



A few possible values of G :

$$G = 7 + (-1) + 9 + 3$$

$$G = 7 + (-1) + 10$$

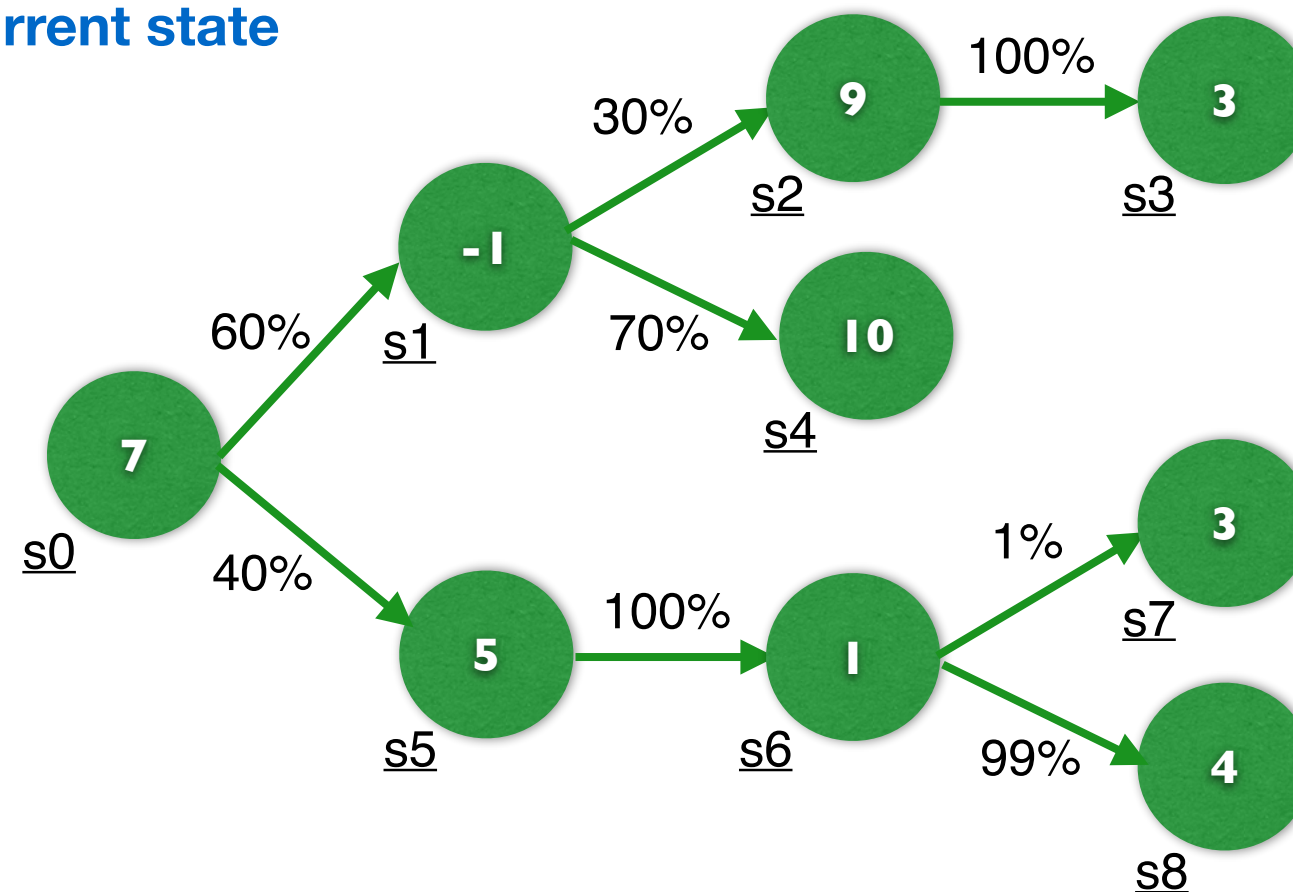
$$G = 7 + 5 + 1 + 3$$

Intuitively, the **expected return** $\mathbb{E}[G | \pi] = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t | \pi \right]$

is the weighted sum of all possible returns
(weighted by their probabilities)

$$G := \sum_{t=0}^{\infty} \gamma^t R_t = R_0 + R_1 + R_2 + \dots$$

G: one possible sum of rewards
if starting at the current state
(a random variable)



A few possible values of G :

$$G = 7 + (-1) + 9 + 3$$

$$G = 7 + (-1) + 10$$

$$G = 7 + 5 + 1 + 3$$

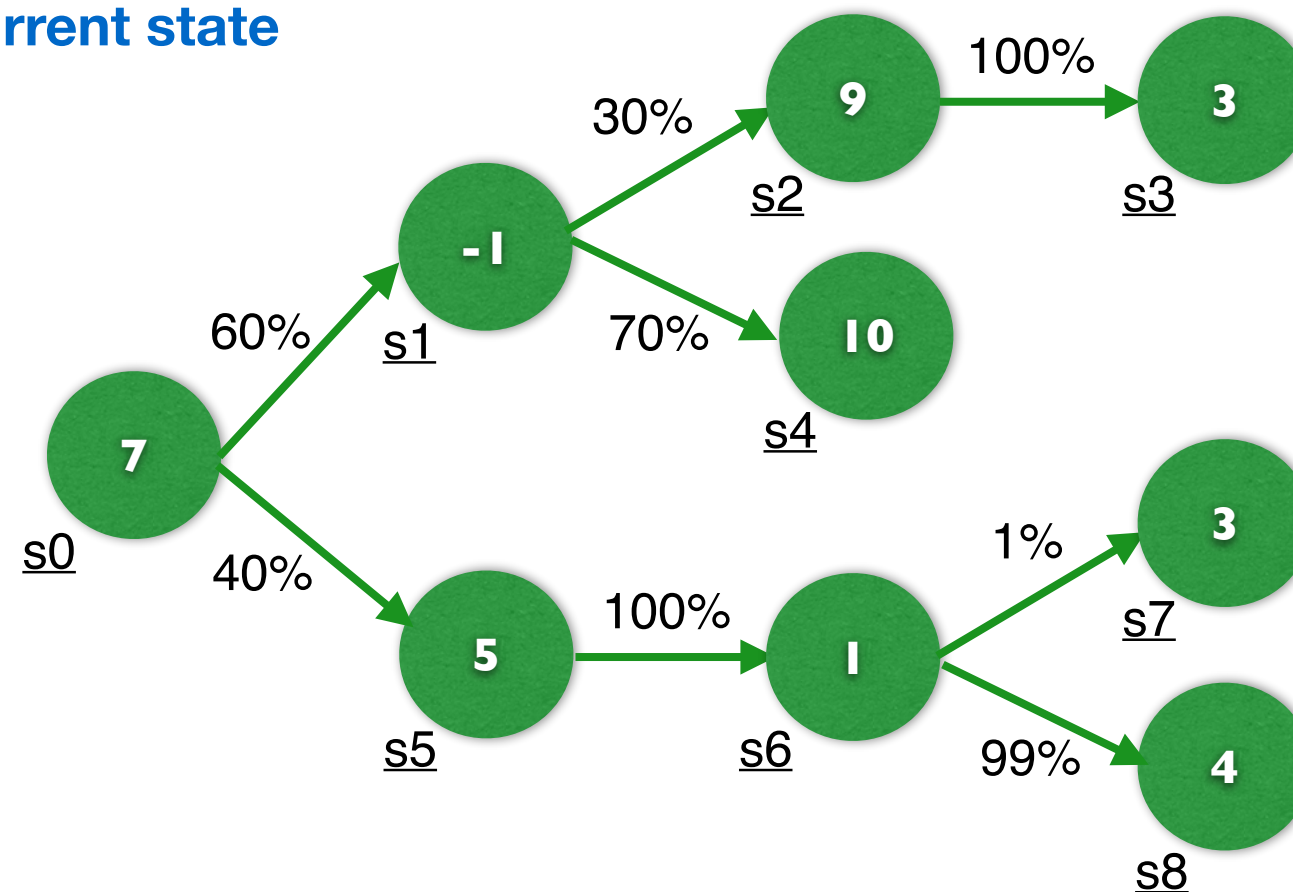
Intuitively, the **expected return** $\mathbb{E}[G | \pi] = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_t | \pi\right]$

How "good" it is to be in some state?

If large, the agent expects to gain lots of rewards from it

$$G := \sum_{t=0}^{\infty} \gamma^t R_t = R_0 + R_1 + R_2 + \dots$$

G: one possible sum of rewards
if starting at the current state
(a random variable)



Intuitively, the expected return $\mathbb{E}[G | \pi] = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t | \pi \right]$

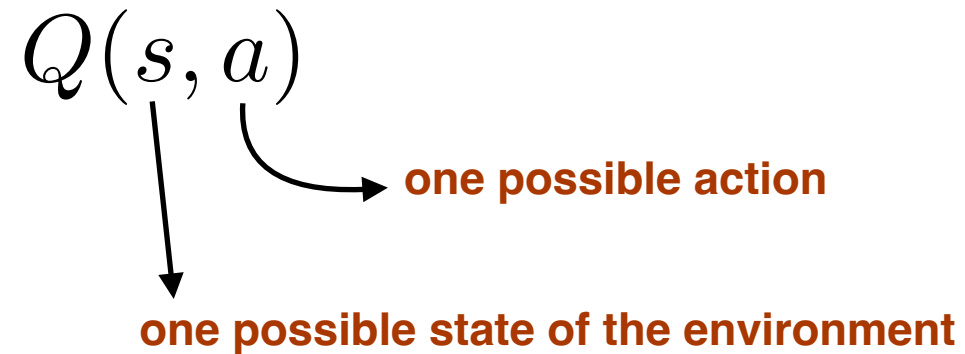
How "good" it is to be in some state?



RL is mostly about learning actions that take the agent to “good” states

Measuring Total Reward The Agent can Get

How much total reward I can achieve
if I start from state s
and execute one particular action a
(not necessarily the best one)



Measuring Total Reward The Agent can Get

$$Q(s, a)$$

How much total reward I can achieve
if I start from state s
and execute one particular action a
(not necessarily the best one)



current state (s) of the game

action $a = \text{RIGHT}$

$$Q(s = \text{[Screenshot]}, a = \text{RIGHT})$$

Total number of points, to the end of the level,
if it goes **RIGHT** in this state

Q-Learning

$$Q_{k+1}(s, a) \leftarrow Q_k(s, a)$$

new estimate of how much
return it can get
if executing action a in state s

current estimate of how much
return it can get
if executing action a in state s

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

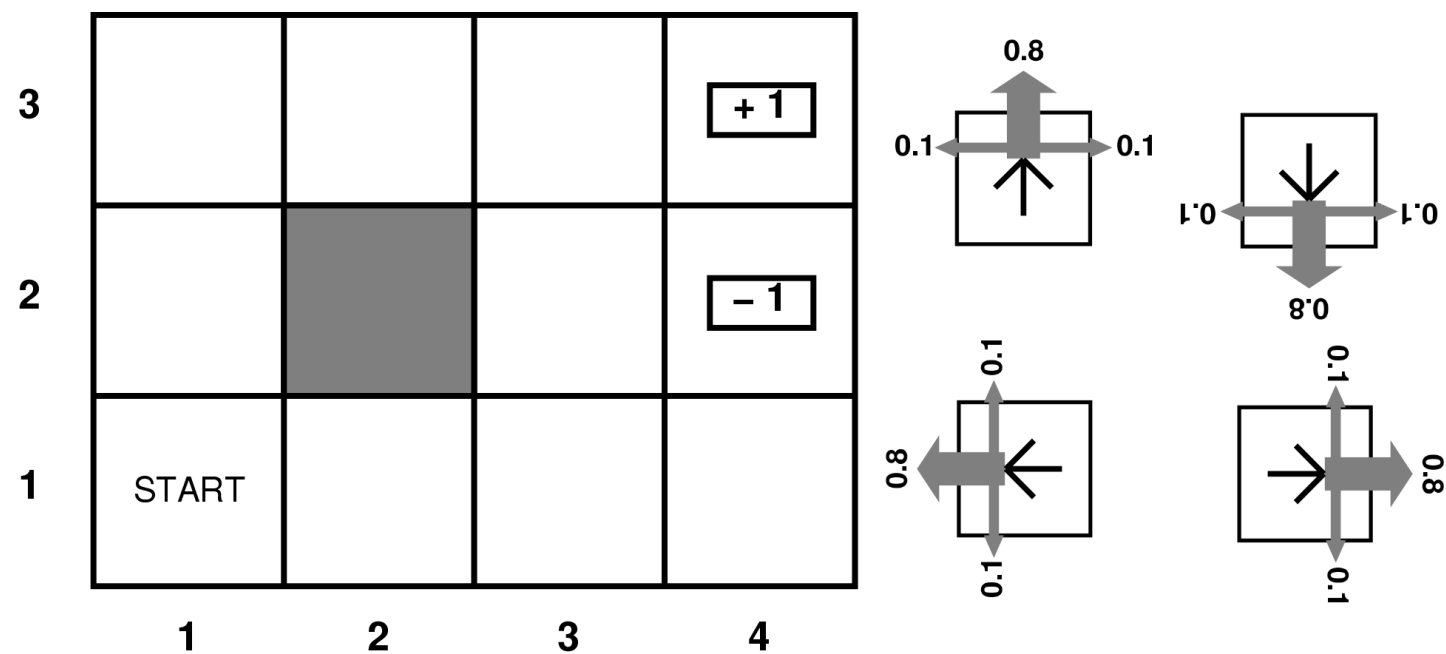
1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning

Q-Learning algorithm

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$



reward **-0.04** after every action
(except when entering the rightmost states)

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+ 1
	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	- 1
	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Initialize the Q table
 - $Q(s,a) \leftarrow$ some number (zero or random), $\forall \mathbf{s}, \mathbf{a}$

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+1
	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	-1
	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Assume the agent was in state (3,3)

- $Q((3,3), \uparrow) = 2.1$
- $Q((3,3), \leftarrow) = 4.2$
- $Q((3,3), \rightarrow) = 3.2$
- $Q((3,3), \downarrow) = 0.21$

$$\pi(s) = \arg \max_a Q(s, a)$$

$$\pi(s) = \leftarrow$$

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+1
	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	-1
	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Assume the agent was in state (3,3)
 - Executed action ←
 - $s = (3,3)$
 - $a = \leftarrow$
 - $r = ?$
 - $s' = ?$

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+ 1
	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	- 1
	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Assume the agent was in state (3,3)
 - Executed action ←
 - $s = (3,3)$
 - $a = \leftarrow$
 - $r = -0.04$
 - $s' = (3,2)$

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

- Assume the agent was in state (3,3)

- Executed action $\leftarrow$

- $s = (3,3)$
- $a = \leftarrow$
- $r = -0.04$
- $s' = (3,2)$

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+ 1
2	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	- 1
1	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

Expected to get 4.2 total reward by executing LEFT

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

- Assume the agent was in state (3,3)

- Executed action $\leftarrow$

- $s = (3,3)$

- $a = \leftarrow$

- $r = -0.04$

- $s' = (3,2)$

Expected to get 4.2 total reward by executing LEFT

But in reality...

it received a first reward of -0.04

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+1
2	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	-1
1	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

- Assume the agent was in state (3,3)

- Executed action $\leftarrow$

- $s = (3,3)$

- $a = \leftarrow$

- $r = -0.04$

- $s' = (3,2)$

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+1
2	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	-1
1	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

Expected to get 4.2 total reward by executing LEFT

But in reality...

it received a first reward of -0.04
and went to a state

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

- Assume the agent was in state (3,3)
 - Executed action $\leftarrow$
 - $s = (3,3)$
 - $a = \leftarrow$
 - $r = -0.04$
 - $s' = (3,2)$

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← 0.3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+1
2	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	-1
1	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

Expected to get 4.2 total reward by executing LEFT

But in reality...

it received a first reward of -0.04 and went to a state from which it thinks it can get 0.6

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

- Assume the agent was in state (3,3)
 - Executed action $\leftarrow$
 - $s = (3,3)$
 - $a = \leftarrow$
 - $r = -0.04$
 - $s' = (3,2)$

3	$\uparrow 0.5$ $\leftarrow 1.7 \ 0.9 \rightarrow$ $\downarrow 1.6$	$\uparrow 0.3$ $\leftarrow .3 \ 0.6 \rightarrow$ $\downarrow 0.1$	$\uparrow 2.1$ $\leftarrow 4.2 \ 3.2 \rightarrow$ $\downarrow 0.21$	+ 1
	$\uparrow 0.1$ $\leftarrow 0.8 \ 0.3 \rightarrow$ $\downarrow 0.32$		$\uparrow 0.52$ $\leftarrow 1.1 \ 0.1 \rightarrow$ $\downarrow 0.93$	- 1
	$\uparrow 0.25$ $\leftarrow 1.0 \ 0.1 \rightarrow$ $\downarrow -0.66$	$\uparrow 1.5$ $\leftarrow 2.3 \ 0.3 \rightarrow$ $\downarrow 0.0$	$\uparrow 0.21$ $\leftarrow 1.2 \ 0.0 \rightarrow$ $\downarrow 1.4$	$\uparrow 0.1$ $\leftarrow 1.0 \ 0.2 \rightarrow$ $\downarrow 1.9$
	1	2	3	4

Expected to get 4.2 total reward by executing LEFT

But in reality...

it received a first reward of -0.04
and went to a state from which it thinks it can get 0.6

Based on this last experience, it now thinks it can get
 $-0.04 + 0.6 = 0.56$ (way less than 4.2!)

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+ 1
	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	- 1
	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Assume the agent was in state (3,3)

- Executed action ←

- $s = (3,3)$
- $a = \leftarrow$
- $r = -0.04$
- $s' = (3,2)$

$$Q_{k+1}(s, a) \leftarrow$$

Should be updated towards
 $-0.04 + 0.6 = 0.56$

$$\left(\underset{\downarrow -0.04}{r} + \max_{a'} \underset{\downarrow 0.6}{Q_k(s', a')} \right)$$

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+ 1
2	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	- 1
1	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Assume the agent was in state (3,3)

- Executed action ←

- $s = (3,3)$

- $a = \leftarrow$

- $r = -0.04$

- $s' = (3,2)$

assume

$$\alpha = 0.5$$

$$\gamma = 1.0$$

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+1
2	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	-1
1	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Assume the agent was in state (3,3)

- Executed action ←

• $s = (3,3)$

• $a = \leftarrow$

• $r = -0.04$

• $s' = (3,2)$

assume
 $\alpha = 0.5$
 $\gamma = 1.0$

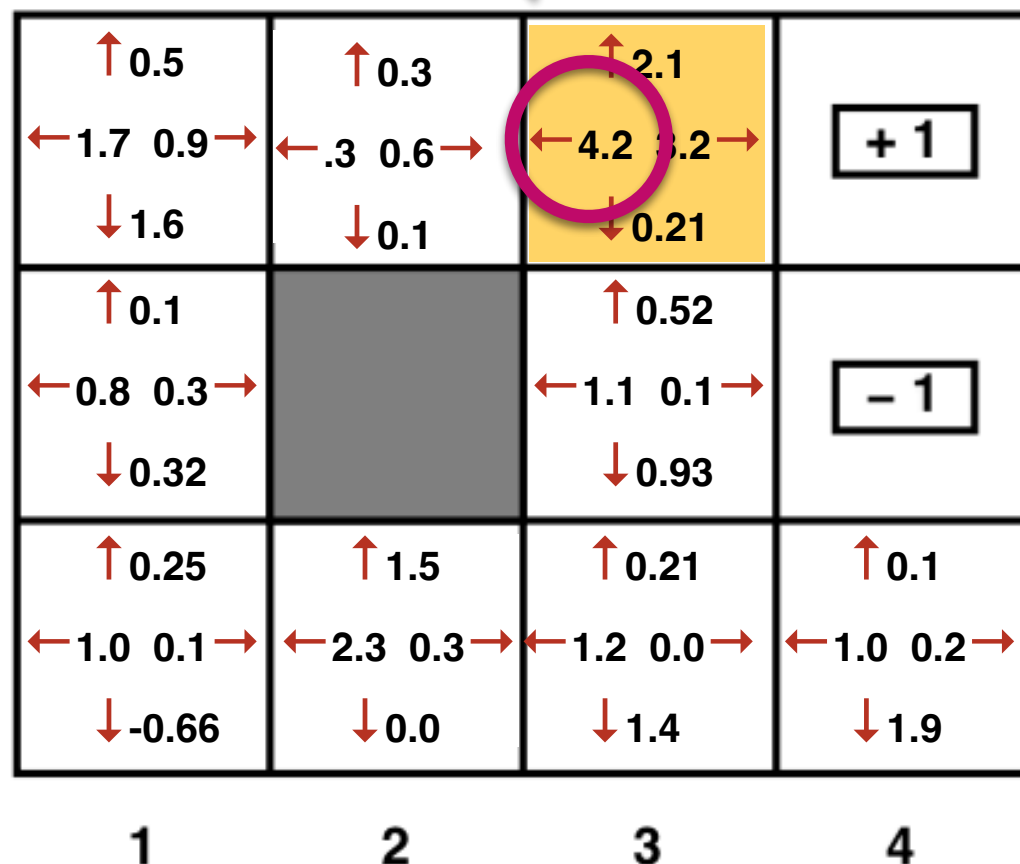
$$\begin{aligned}
 Q((3,3), \leftarrow) &\leftarrow (1 - \alpha) Q((3,3), \leftarrow) + \alpha \left(-0.04 + \gamma \max_{a'} Q((3,2), a') \right) \\
 &\leftarrow (1 - \alpha) 4.2 + \alpha \left(-0.04 + \gamma 0.6 \right) \\
 &\leftarrow 0.5 * 4.2 + \alpha \left(-0.04 + 1.0 * 0.6 \right) = \mathbf{2.38}
 \end{aligned}$$

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm



3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 0.2 → ↓ 0.21	+1
2	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	-1
1	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Assume the agent was in state (3,3)

- Executed action ←

- $s = (3,3)$

- $a = \leftarrow$

- $r = -0.04$

- $s' = (3,2)$

assume
 $\alpha = 0.5$
 $\gamma = 1.0$

$$\begin{aligned}
 Q((3,3), \leftarrow) &\leftarrow (1 - \alpha) Q((3,3), \leftarrow) + \alpha \left(-0.04 + \gamma \max_{a'} Q((3,2), a') \right) \\
 &\leftarrow (1 - \alpha) 4.2 + \alpha \left(-0.04 + \gamma 0.6 \right) \\
 &\leftarrow 0.5 * 4.2 + \alpha \left(-0.04 + 1.0 * 0.6 \right) = \mathbf{2.38}
 \end{aligned}$$

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 2.38 3.2 → ↓ 0.21	+ 1
2	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	- 1
1	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Assume the agent was in state (3,3)

- Executed action ←

- $s = (3,3)$

- $a = \leftarrow$

- $r = -0.04$

- $s' = (3,2)$

assume
 $\alpha = 0.5$
 $\gamma = 1.0$

$$\begin{aligned}
 Q((3,3), \leftarrow) &\leftarrow (1 - \alpha) Q((3,3), \leftarrow) + \alpha \left(-0.04 + \gamma \max_{a'} Q((3,2), a') \right) \\
 &\leftarrow (1 - \alpha) 4.2 + \alpha \left(-0.04 + \gamma 0.6 \right) \\
 &\leftarrow 0.5 * 4.2 + \alpha \left(-0.04 + 1.0 * 0.6 \right) = \mathbf{2.38}
 \end{aligned}$$

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 2.38 3.2 → ↓ 0.21	+ 1
	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	- 1
	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Agent is now in state (2,3)

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 2.38 3.2 → ↓ 0.21	+ 1
	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	- 1
	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Agent is now in state (2,3)

(...)
process repeats

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

3	↑ 0.5 ← 1.7 0.9 → ↓ 1.6	↑ 0.3 ← .3 0.6 → ↓ 0.1	↑ 2.1 ← 4.2 3.2 → ↓ 0.21	+ 1
	↑ 0.1 ← 0.8 0.3 → ↓ 0.32		↑ 0.52 ← 1.1 0.1 → ↓ 0.93	- 1
	↑ 0.25 ← 1.0 0.1 → ↓ -0.66	↑ 1.5 ← 2.3 0.3 → ↓ 0.0	↑ 0.21 ← 1.2 0.0 → ↓ 1.4	↑ 0.1 ← 1.0 0.2 → ↓ 1.9
	1	2	3	4

- Problem: when agent was in state (3,3)
 - Executed action ←
 - $s = (3,3)$
 - $a = \leftarrow$
- Selected action that seemed the best (highest Q)
- But during learning, Q's might be wrong!
- Agent doesn't know yet which actions are best
- Should it always execute the one that seems the best?

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

1. Agent is in s , picks action a .
2. Receives reward r and observes next state s'
3. Updates $Q_k \rightarrow Q_{k+1}$

Q-Learning algorithm

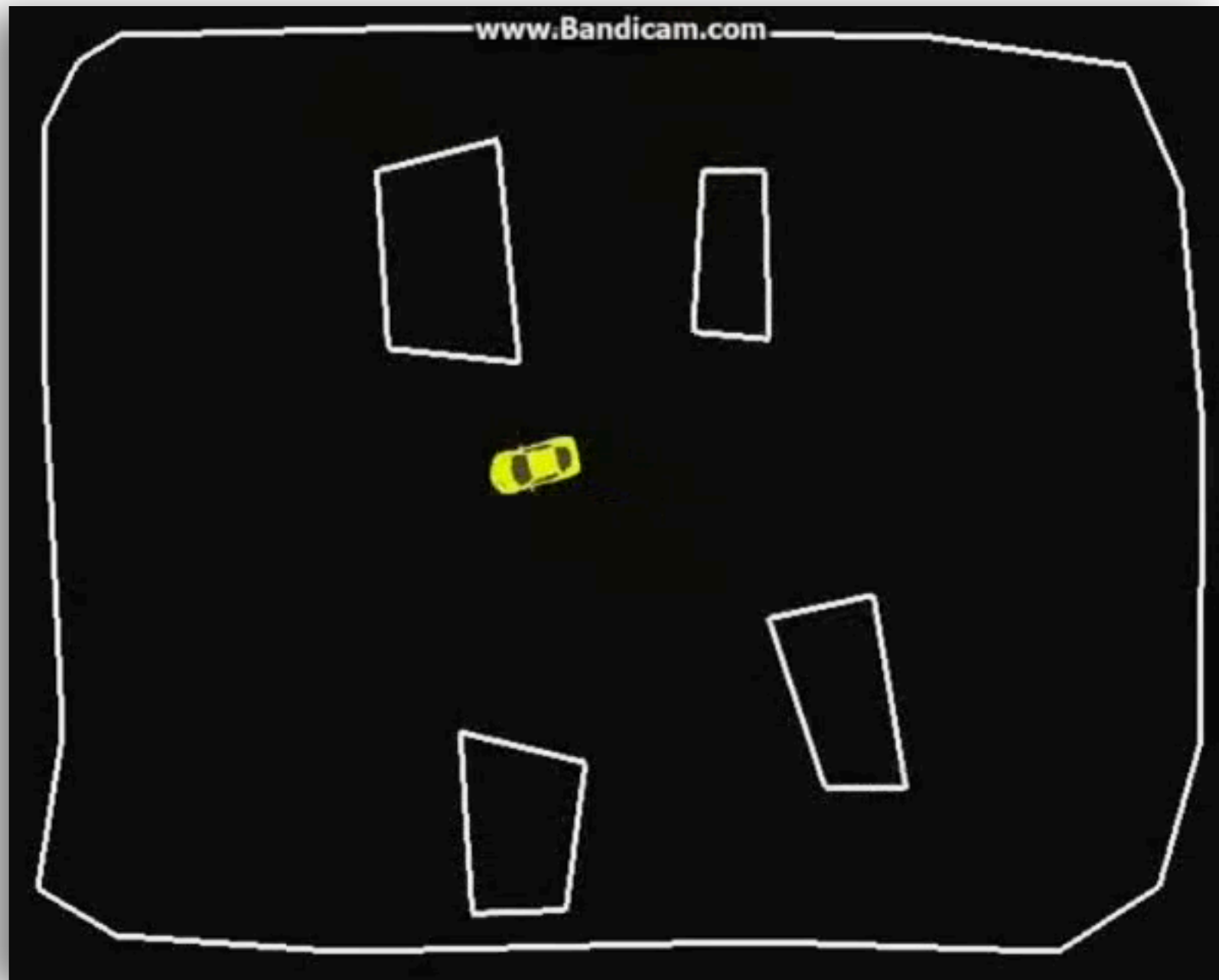
- How to select an action
 - Usually pick the one that seems the best, but also try out new actions!
 - e.g.
 - execute the action with highest Q-value 90% of the time
 - execute a random action 10% of the time

Initially agent doesn't know environment/outcome of actions → Exploration

Q-Learning

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha \left(r + \gamma \max_{a'} Q_k(s', a') \right)$$

Learning to drive



<https://www.youtube.com/watch?v=zOgSC---rgM>

Reinforcement Learning

Learning to land on the moon



https://www.youtube.com/watch?v=_pF1Un5zvp4

Reinforcement Learning

Learning to play ping-pong



<https://www.youtube.com/watch?v=SH3bADiB7uQ>



Machine Learning

CMPSCI 589

Bruno C. da Silva

bsilva@cs.umass.edu

Reinforcement Learning (part 2/3)

UMassAmherst
College of Information
& Computer Sciences



This work is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nc-nd/4.0/)

